

CSE 241 OOP HOMEWORK 1 PRESENTATION DOCUMENT

In this pdf document you will find explanations of the homework I did, lines of code, Uml table of classes and many explanations.

Description of code fragments:

main.cpp:

```
1 #include "AnsiTerminal.h"
2 #include "Table.h"
3 #include "Place.h"
4 #include <iostream>
5
6 int main() {
7     AnsiTerminal terminal; // AnsiTerminal nesnesi oluşturuluyor
8     Table table(10, 10); // 10x10 boyutunda bir tablo oluşturuluyor
9     Place place(table, terminal); // Place nesnesi oluşturuluyor, tablo ve terminal parametre olarak alınıyor
10
11     terminal.clearScreen(); // Ekranı temizle
12     table.display(); // Tabloyu ekranda göster
13     terminal.printInvertedAt(1, 1, "*"); // (1, 1) koordinatına ters renkli '*' karakteri yazdır
14
15     place.navigate(); // Kullanıcıyı yönlendirmek için 'navigate' fonksiyonu çağırılıyor
16
17     terminal.clearScreen(); // İşlem tamamlandıktan sonra ekran temizleniyor
18     return 0; // Program başarıyla sonlandırılıyor
19 }
```

As you can see above, in the main.cpp file, you can see the code that creates the table and gets the number of rows and columns of the table. As a result, we create the table and get the number of rows and columns using other .h libraries in the main.cpp file.

Cell.h:

```
1 #ifndef CELL_H
2 #define CELL_H
3
4 #include <string>
5
6 class Cell {
7 private:
8     std::string value; // Hücrenin hesaplanan değeri (string olarak saklanır)
9     std::string formula; // Hücrede yer alan formül (varsa)
10
11     bool isFormula; // Hücrenin bir formül içerip içermediğini belirtir
12
13 public:
14     Cell() : value(""), formula(""), isFormula(false) {}
15
16     // Getter ve Setter fonksiyonları
17     const std::string& getValue() const { return value; }
18     void setValue(const std::string& newValue) {
19         value = newValue;
20         /* Eğer yeni değer bir formül değilse (örneğin, '-' ile başlanıyorsa),
21            hücre formül hücresi olarak işaretlenmez. */
22     }
23
24     const std::string& getFormula() const { return formula; }
25     void setFormula(const std::string& newFormula) {
26         formula = newFormula;
27         isFormula = true; // Yeni bir formül atanıyorsa, bu bir formül hücresidir
28     }
29
30     // Hücrenin bir formül içerip içermediğini döner
31     bool isFormulaCell() const { return isFormula; }
32 };
33
34 #endif // CELL_H
35
```

Since the Cell.h file is short, we did not need to create a .cpp file, we did the implementations directly in the file.

This file defines a Cell class representing a cell in a spreadsheet. The class contains the following:

Private Attributes:

value: Stores the calculated value of the cell as a string.

formula: Stores the formula inside the cell, if any.

isFormula: A boolean flag indicating whether the cell contains a formula.

Public Methods:

Constructor: Initializes the cell with an empty value, no formula, and sets isFormula to false.

Getters and Setters:

getValue() and setValue(): Retrieve and set the value of the cell.

getFormula() and setFormula(): Retrieve and set the formula of the cell. If a formula is set, the cell is marked as containing a formula.

isFormulaCell(): Returns a boolean indicating whether the cell contains a formula.

This class allows handling both regular values and formulas in a cell, while distinguishing between the two.

Table.h:

```
1 #ifndef TABLE_H
2 #define TABLE_H
3
4 #include <vector> // Dinamik dizi (vector) kullanabilmek için gerekli başlık dosyası
5 #include <string> // String sınıfı için gerekli başlık dosyası
6 #include <iostream> // Giriş/çıkış işlemleri için gerekli başlık dosyası (örn: std::cout, std::cin)
7 #include "Cell.h" // Hücre sınıfı başlık dosyasını dahil eder
8 #include <map> // Harita (map) veri yapısını kullanabilmek için gerekli başlık dosyası
9
10 // Table sınıfı, hücrelerden oluşan bir tabloyu temsil eder
11 class Table {
12 private:
13     std::vector<std::vector<Cell>> grid; // Hücrelerden oluşan 2D tablo (satır ve sütunlar)
14     int rowCount; // Tablo satır sayısı
15     int colCount; // Tablo sütun sayısı
16
17     // Hücre referansını çözümleyerek satır ve sütun indeksini döner
18     // powered by ChatGPT
19     std::pair<int, int> parseCellReference(const std::string& cellRef) const;
20
21     // Formül girdisini temizler (şu an için sadece kopyalar, gelecekte düzenleme yapılabilir)
22     std::string cleanFormulaInput(const std::string& input);
23
24     // Sütun harfini, sütun indeksine dönüştürür (A -> 0, B -> 1, ...)
25     int columnToIndex(const std::string& col) const;
26 }
```

```

15     int columnToIndex(const std::string& col) const;
16
17 public:
18     // Constructor: Tabloyu satır ve sütun sayısı ile başlatır
19     Table(int rows, int cols);
20
21     // Formülleri günceller (tablodaki formül hücrelerini değerlendirir)
22     void updateFormulas();
23
24     // Satır sayısını döner
25     int getRowCount() const { return rowCount; }
26
27     // Sütun sayısını döner
28     int getColCount() const { return colCount; }
29
30     // Verilen hücrenin formülünü döner
31     std::string getCellFormula(int currRow, int currCol) const { return grid[currRow][currCol].getFormula(); }
32
33     // Hücreye veri ekler
34     void setCell(int row, int col, const std::string& value);
35
36     // Hücredeki değeri döner
37     std::string getCell(int row, int col) const;
38
39     // Hücre referansına karşılık gelen değeri döner (örneğin: "A1")
40     std::string getCellByReference(const std::string& reference) const;
41
42     // Verilen hücrenin formülünü döner
43     std::string getCellFormula(int currRow, int currCol) const { return grid[currRow][currCol].getFormula(); }
44
45     // Hücreye veri ekler
46     void setCell(int row, int col, const std::string& value);
47
48     // Hücredeki değeri döner
49     std::string getCell(int row, int col) const;
50
51     // Hücre referansına karşılık gelen değeri döner (örneğin: "A1")
52     std::string getCellByReference(const std::string& reference) const;
53
54     // Formülü değerlendirir ve sonucu döner
55     // powered by ChatGPT
56     double evaluateFormula(const std::string& formula, int currentRow, int currentCol);
57
58     // Tabloyu ekrana yazdırır
59     void display() const;
60
61     // Hücrenin geçerli olup olmadığını kontrol eder
62     bool isValidCell(const std::string& reference) const;
63 };
64 #endif // TABLE_H

```

This file defines a Table class representing a spreadsheet:

Attributes:

grid: A 2D vector of Cell objects.

rowCount and colCount: Store the number of rows and columns.

Methods:

Constructor: Initializes the table with specified rows and columns.

updateFormulas(): Evaluates and updates the formulas in the table.

Getters: Returns row count, column count, and cell formulas.

setCell() / getCell(): Set and get cell values.

getCellByReference(): Retrieve cell value by its reference (e.g., "A1").

evaluateFormula(): Evaluates a formula and returns the result.

display(): Prints the table.

isValidCell(): Checks if a cell reference is valid.

The class handles a grid of cells, evaluates formulas, and manages cell references.

Table.cpp:

```
1 #include "Table.h" // Table sınıfının başlık dosyasını dahil eder
2 #include <cmath> // Matematiksel işlemler için gerekli
3 #include <sstream> // String akışları (ostringstream, istream) için gerekli başlık dosyası
4 #include <iostream>
5 #include <stdexcept> // Hata işleme (exception handling) için gerekli başlık dosyası
6
7 // Constructor: Satır ve sütun sayısına göre tabloyu başlatır
8 Table::Table(int rows, int cols) : rowCount(rows), colCount(cols) {
9     grid.resize(rowCount, std::vector<Cell>(colCount)); // Tabloyu boyutlara göre yeniden boyutlandırır
10 }
11
12 // Formül girdisini temizler (şu an için sadece kopyalar, gelecekte eklenebilir)
13 std::string Table::cleanFormulaInput(const std::string& input) {
14     std::string cleaned = input;
15     return cleaned; // Girdiği olduğu gibi döner
16 }
17
18 // Sütun harfini, sütun indeksine dönüştürür (A -> 0, B -> 1, ...)
19 int Table::columnToIndex(const std::string& col) const {
20     return col[0] - 'A'; // Harften sayıya dönüşüm
21 }
22
23 // Tabloyu günceller ve formülleri hesaplar
24 void Table::updateFormulas() {
25     for (int row = 0; row < rowCount; ++row) {
26         for (int col = 0; col < colCount; ++col) {
27             if (grid[row][col].isFormulaCell()) { // Hücre formül içeriyorsa
28                 std::string formula = grid[row][col].getFormula();
29                 std::string cleanFormula = cleanFormulaInput(formula); // Formülü temizle
30                 double result = evaluateFormula(cleanFormula, row, col); // Formülü hesapla
31                 grid[row][col].setValue(std::to_string(result)); // Hesaplanan sonucu hücreye ata
32             }
33         }
34     }
35 }
36
37 // Formülü değerlendirir ve sonucu döner
38 // powered by ChatGPT
39 double Table::evaluateFormula(const std::string& formula, int currentRow, int currentCol) {
40     std::istringstream iss(formula);
41     double lhs, rhs; // Sol ve sağ operandlar
42     char op; // İşlem operatörü
43
44     iss >> lhs; // İlk sayıyı al
45     while (iss >> op >> rhs) { // İşlem sırasını uygula
46         switch (op) {
47             case '+': lhs += rhs; break; // Toplama
48             case '-': lhs -= rhs; break; // Çıkarma
```

```

50     case '/':
51         if (rhs == 0) throw std::invalid_argument("Cant divide by zero"); // Bölme sıfıra yapılırsa l
52         lhs /= rhs; // Bölme
53         break;
54     default: throw std::invalid_argument("Invalid operator"); // Geçersiz operatör
55 }
56 }
57
58 return lhs; // Sonucu döner
59 } // powered by ChatGPT
60
61 // Tabloyu ekrana yazdırır
62 void Table::display() const {
63     for (int col = 0; col < colCount; ++col) {
64         std::cout << static_cast<char>('A' + col) << "\t"; // Sütun başlıklarını yazdır
65     }
66     std::cout << std::endl;
67
68     for (int row = 0; row < rowCount; ++row) {
69         std::cout << row + 1 << "\t"; // Satır numarasını yazdır
70         for (int col = 0; col < colCount; ++col) {
71             std::cout << getCell(row, col) << "\t"; // Hücredeki değeri yazdır
72         }
73         std::cout << std::endl;
74     }
75 }
76
77 // Verilen hücreye ait değeri döner
78 std::string Table::getCell(int row, int col) const {
79     return grid[row][col].getValue(); // Hücre değerini döner
80 }
81
82 // Hücre referansının geçerli olup olmadığını kontrol eder
83 bool Table::isValidCell(const std::string& reference) const {
84     auto [row, col] = parseCellReference(reference); // Referansı çözümü
85     return (row >= 0 && row < rowCount && col >= 0 && col < colCount); // Geçerli satır ve sütun olup olmadı
86     kontrol et
87 }
88 // Hücre referansına karşılık gelen değeri döner
89 std::string Table::getCellByReference(const std::string& reference) const {
90     auto [row, col] = parseCellReference(reference); // Referansı çözümü
91     return grid[row][col].getValue(); // Hücre değerini döner
92 }
93
94 // Hücre referansını satır ve sütun indekslerine dönüştürür
95 // powered by ChatGPT
96 std::pair<int, int> Table::parseCellReference(const std::string& cellRef) const {
97     int col = columnToIndex(cellRef.substr(0, 1)); // Sütun harfini indeks olarak dönüştür
98     int row = std::stoi(cellRef.substr(1)) - 1; // Satır numarasını indeks olarak dönüştür
99     return {row, col}; // Satır ve sütun indeksini döner
100 } // powered by ChatGPT

```

As you can see in table.cpp, table.h has implementations and is supported with comment lines.

double Table::evaluateFormula(const std::string& formula, int currentRow, int currentCol) and

I wrote std::pair<int, int> Table::parseCellReference(const std::string& cellRef) const with help from Chatgpt

Place.h:

```
1 #ifndef PLACE_H
2 #define PLACE_H
3
4 #include "Table.h"
5 #include "AnsiTerminal.h"
6
7 // Place sınıfı, kullanıcıyı tablo üzerinde yönlendiren sınıftır
8 class Place {
9 private:
10     Table& table; // Tabloyu tutacak referans
11     AnsiTerminal& terminal; // Terminali tutacak referans
12     int currentRow; // Mevcut satır numarası
13     int currentCol; // Mevcut sütun numarası
14
15 public:
16     // Constructor: Table ve AnsiTerminal nesneleri ile başlangıç değerlerini ayarlayan fonksiyon
17     Place(Table& sheet, AnsiTerminal& term);
18
19     // Kullanıcıyı tablo üzerinde yönlendiren fonksiyon
20     void navigate();
21 };
22
23 #endif // PLACE_H
24
```

This file defines a Place class that helps navigate a table:

- **Attributes:**
 - table: A reference to the Table object.
 - terminal: A reference to the AnsiTerminal object.
 - currentRow and currentCol: Store the current position of the user.
- **Methods:**
 - **Constructor:** Initializes the table and terminal references.
 - **navigate():** Guides the user through the table.

The class facilitates navigation within the table and terminal.

Place.cpp:

```
#include "Place.h"
#include <iostream>

// Constructor: Table ve AnsiTerminal nesneleri ile birlikte başlangıç değerlerini ayarlıyor
Place::Place(Table& sheet, AnsiTerminal& term)
    : table(sheet), terminal(term), currentRow(1), currentCol(1) {}

// Kullanıcıyı tablodaki hücrelerde yönlendiren fonksiyon
void Place::navigate() {
    char key; // Kullanıcının tuş girişini tutacak değişken

    // Sonsuz döngü, kullanıcı 'q' tuşuna basana kadar devam eder
    while (true) {
        terminal.clearScreen(); // Ekranı temizle
        table.display(); // Tabloyu ekranda göster
        terminal.printInvertedAt(currentRow, currentCol, "*"); // Şu anki konumu '*' ile işaretle

        key = terminal.getSpecialKey(); // Kullanıcının tuş girişini al

        terminal.printAt(currentRow, currentCol, " "); // Önceki işareti kaldır

        // Kullanıcının tuşuna göre hareket et
        switch (key) {
            case 'U': // Yukarı hareket
                if (currentRow > 1) currentRow--; // İlk satıra ulaşılmadıysa yukarı git
                break;
            case 'D': // Aşağı hareket
                if (currentRow < table.getRowCount()) currentRow++; // Son satıra ulaşılmadıysa aşağı git
                break;
            case 'R': // Sağa hareket
                if (currentCol < table.getColCount()) currentCol++; // Son sütuna ulaşılmadıysa sağa git
                break;
            case 'L': // Sol'a hareket
                if (currentCol > 1) currentCol--; // İlk sütuna ulaşılmadıysa sola git
                break;
            case 'q': // 'q' tuşuna basıldığında çık
                return;
        }
    }
}
```

Place.cpp contains the implementation of the place.h file you saw above.

I have written these codes correctly and they work in harmony with each other, but although I have been trying for days, I could not reach the result I wanted in the

terminal. I can print the table I want to the terminal, I can navigate between the values in the table, but I cannot get values from the user to the table. You can access the terminal output below.

```
fatih@fatih:~/Masaüstü/andthelastone$ g++ -o deneme AnsiTerminal.cpp main.cpp Place.cpp Table.cpp
fatih@fatih:~/Masaüstü/andthelastone$ ./deneme
```

A	B	C	D	E	F	G	H	I	J
1									
2	*								
3									
4									
5									
6									
7									
8									
9									
10									

UML Class Diagram:

